

abmodulex

June 6, 2026

compile_options	-lang cpp -fpga-mem-th 4 -ct 1 -es 1 -mcd 16 -mdd 1024 -mdy 33 -single -
filename	abmodulex.dsp
name	abmodulex
seam.ambisonics.lib/author	Giuseppe Silvi
seam.ambisonics.lib/license	CC3
seam.ambisonics.lib/name	SEAM Ambisonics Library
seam.ambisonics.lib/version	0.1
seam.lib/author	Giuseppe Silvi, Davide Tedesco
seam.lib/license	CC3
seam.lib/name	Faust SEAM main lib
seam.lib/version	0.2

This document provides a mathematical description of the Faust program text stored in the `abmodulex.dsp` file. See the notice in Section 3 (page 2) for details.

1 Mathematical definition of process

The *abmodulex* program evaluates the signal transformer denoted by `process`, which is mathematically defined as follows:

1. Output signals y_i for $i \in [1, 4]$ such that

$$\begin{aligned}y_1(t) &= 0.5 \cdot (x_4(t) + x_3(t) + s_1(t)) \\y_2(t) &= 0.5 \cdot (x_1(t) + x_4(t) - x_2(t) + x_3(t)) \\y_3(t) &= 0.5 \cdot (x_1(t) + x_3(t) - x_2(t) + x_4(t)) \\y_4(t) &= 0.5 \cdot (s_1(t) - x_3(t) + x_4(t))\end{aligned}$$

2. Input signals x_i for $i \in [1, 4]$
3. Intermediate signal s_1 such that

$$s_1(t) = x_1(t) + x_2(t)$$

2 Block diagram of process

The block diagram of `process` is shown on Figure 1 (page 2).

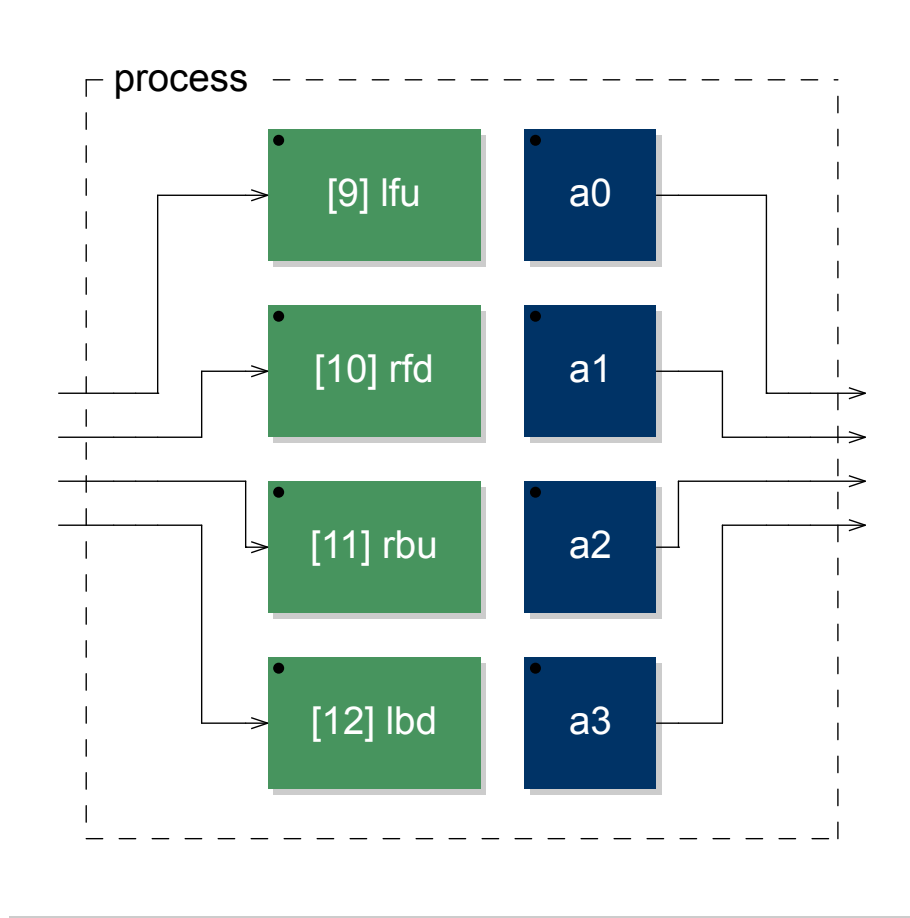


Figure 1: Block diagram of `process`

3 Notice

- This document was generated using Faust version 2.85.5 on June 06, 2026.
- The value of a Faust program is the result of applying the signal transformer denoted by the expression to which the `process` identifier is bound to input signals, running at the f_s sampling frequency.
- Faust (*Functional Audio Stream*) is a functional programming language

designed for synchronous real-time signal processing and synthesis applications. A Faust program is a set of bindings of identifiers to expressions that denote signal transformers. A signal s in S is a function mapping¹ times $t \in \mathbb{Z}$ to values $s(t) \in \mathbb{R}$, while a signal transformer is a function from S^n to S^m , where $n, m \in \mathbb{N}$. See the Faust manual for additional information (<http://faust.grame.fr>).

- Every mathematical formula derived from a Faust expression is assumed, in this document, to having been normalized (in an implementation-dependent manner) by the Faust compiler.
- A block diagram is a graphical representation of the Faust binding of an identifier I to an expression E ; each graph is put in a box labeled by I . Subexpressions of E are recursively displayed as long as the whole picture fits in one page.
- The `abmodulex-mdoc/` directory may also include the following subdirectories:
 - `cpp/` for Faust compiled code;
 - `pdf/` which contains this document;
 - `src/` for all Faust sources used (even libraries);
 - `svg/` for block diagrams, encoded using the Scalable Vector Graphics format (<http://www.w3.org/Graphics/SVG/>);
 - `tex/` for the L^AT_EX source of this document.

4 Faust code listings

This section provides the listings of the Faust code used to generate this document, including dependencies.

Listing 1: `abmodulex.dsp`

```

1 // SEAM-LTM · ABMODULEX -Faust spec / block-diagram source.
2 //
3 // Canonical DSP: sam.abmodulex in seam.ambisonics.lib
4 // A-format tetrahedral mic (LFU,RFD,RBU,LBD) -> First-order AmbiX (ACN).
5 // Inverse of bamodulex; the matrix is involutory. The C++ plugin
6 // re-implements this by hand (see plugins/abmodulex/source).
7 //
8 // Regenerate diagrams:
9 // ../../tools/gen-faust-doc.sh abmodulex
10 //
11 import("seam.lib");
12 process = sam.abmodulex;
```

¹Faust assumes that $\forall s \in S, \forall t \in \mathbb{Z}, s(t) = 0$ when $t < 0$.

Listing 2: seam.lib

```

1 declare name "Faust SEAM main lib";
2 declare version "0.2";
3 declare author "Giuseppe Silvi";
4 declare author "Davide Tedesco";
5 declare license "CC3";
6 // calling standard faust libraries
7 import("stdfaust.lib");
8 // ### standard faust lib extensions
9 san = library("seam.analyzers.lib");
10 sba = library("seam.basic.lib");
11 sma = library("seam.math.lib");
12 sfi = library("seam.filters.lib");
13 sre = library("seam.reverbs.lib");
14 sno = library("seam.noises.lib");
15 // ### general Ambisonics theory
16 sam = library("seam.ambisonics.lib");
17 sdw = library("seam.dwt.lib");
18 // ### stereophony and perception
19 sst = library("seam.stereophony.lib");
20 // shr = library("seam.hrtf.lib");
21 // ### CSound and MaxMsp object cloning
22 scs = library("seam.csound.lib");
23 scy = library("seam.cyclone.lib");
24 // ### author specific literature
25 // sds = library("seam.discipio.lib");
26 smg = library("seam.gerzon.lib");
27 // sln = library("seam.nono.lib");
28 // sgn = library("seam.nottoli.lib");
29 sjm = library("seam.moorer.lib");
30 scr = library("seam.roads.lib");
31 sms = library("seam.schroeder.lib");
32 sfv = library("seam.freeverb.lib");
33
34 sff = library("seam.ffunctions.lib");
35
36 // ### live electronics stuff
37 // sgu = library("seam.gui.lib");
38 // shw = library("seam.hardware.lib");
39 // smx = library("seam.mixer.lib");
40
41 // ### instrument specific literature
42 // svc = library("seam.vcs3.lib");

```

Listing 3: stdfaust.lib

```

1 //##### stdfaust.lib #####
2 // The purpose of this library is to give access to all the Faust standard libraries
3 // through a series of environments.
4 //#####
5
6 aa = library("aanl.lib");
7 sf = library("all.lib");
8 an = library("analyzers.lib");
9 ba = library("basics.lib");
10 co = library("compressors.lib");
11 db = library("debug.lib");
12 de = library("delays.lib");
13 dm = library("demos.lib");
14 dx = library("dx7.lib");
15 en = library("envelopes.lib");
16 fd = library("fds.lib");
17 fi = library("filters.lib");

```

```

18 ho = library("hoa.lib");
19 hy = library("hysteresis.lib");
20 it = library("interpolators.lib");
21 la = library("linearalgebra.lib");
22 ma = library("maths.lib");
23 mi = library("mi.lib");
24 mo = library("motion.lib");
25 ef = library("misceffects.lib");
26 no = library("noises.lib");
27 os = library("oscillators.lib");
28 pf = library("phaflangers.lib");
29 pl = library("platform.lib");
30 pm = library("physmodels.lib");
31 qu = library("quantizers.lib");
32 rm = library("reducemaps.lib");
33 re = library("reverbs.lib");
34 ro = library("routes.lib");
35 sp = library("spats.lib");
36 si = library("signals.lib");
37 so = library("soundfiles.lib");
38 sy = library("synths.lib");
39 ve = library("vaeffects.lib");
40 vl = library("version.lib");
41 wa = library("webaudio.lib");
42 wd = library("wdmodels.lib");

```

Listing 4: seam.ambisonics.lib

```

1 declare name "SEAM Ambisonics Library";
2 declare version "0.1";
3 declare author "Giuseppe Silvi";
4 declare license "CC3";
5 //
6 // SEAM Ambisonics Library
7 //
8 // Functions for first-order Ambisonics processing in ACN/SN3D (AmbiX)
9 // format, including format conversion from B-format (FuMa) and
10 // 3D rotation.
11 //
12 // Channel ordering follows the ACN convention:
13 // a0 = W (omnidirectional)
14 // a1 = Y (front-back, left-right)
15 // a2 = Z (up-down)
16 // a3 = X (front-back)
17 //
18 // References:
19 // - Furse-Malham, "A 3-D Sound Primer" (1998)
20 // - Nachbar et al., "ambiX -- A Suggested Ambisonics Format" (2011)
21 // - Gerzon, "Ambisonics. Part two: Studio techniques" (1975)
22 // - Malham, "3-D Sound Spatialization using Ambisonic Techniques" (1999)
23 //
24 //
25 //
26 import("seam.lib");
27 //
28 sam = library("seam.ambisonics.lib");
29 //
30 // B-format to AmbiX conversion
31 //
32 // Converts a first-order Ambisonics signal from B-format (FuMa)
33 // to AmbiX (ACN/SN3D).
34 //
35 // B-format (FuMa): W, X, Y, Z -- with W attenuated by 1/sqrt(2)
36 // AmbiX (ACN/SN3D): W, Y, Z, X -- with W at unity gain

```

```

37 //
38 // The conversion performs two operations:
39 // 1. Rescale W by sqrt(2) to restore unity gain
40 // 2. Reorder channels from FuMa (WXYZ) to ACN (WYZX)
41 //
42 btox(w,x,y,z) = a0, a1, a2, a3
43 with {
44     a0 = w * sqrt(2); // W: restore unity gain
45     a1 = y;           // Y: FuMa -> ACN reorder
46     a2 = z;           // Z: unchanged
47     a3 = x;           // X: FuMa -> ACN reorder
48 };
49 //
50 //           AmbiX -> tetrahedral decoder
51 //
52 // Decodes a first-order AmbiX signal to four loudspeakers placed at
53 // the vertices of a tetrahedron inscribed in a cube:
54 //
55 // LFU -- Left Front Up    ( +Y +Z +X )
56 // RFD -- Right Front Down ( -Y -Z +X )
57 // RBU -- Right Back Up    ( -Y +Z -X )
58 // LBD -- Left Back Down   ( +Y -Z -X )
59 //
60 // Each output is the sum of W and the three Cartesian components
61 // signed according to the corner's position, scaled by 1/2:
62 //
63 // LFU = (a0 + a1 + a2 + a3) / 2
64 // RFD = (a0 - a1 - a2 + a3) / 2
65 // RBU = (a0 - a1 + a2 - a3) / 2
66 // LBD = (a0 + a1 - a2 - a3) / 2
67 //
68 // The matrix is the AmbiX-domain analogue of Gerzon's `bamodule`
69 // (FuMa B-format -> tetrahedron) found in seam.gerzon.lib. The two
70 // differ only in the input reference system; the same diagonal-cube
71 // vertex layout is preserved.
72 //
73 // Compensation shelving filters (Gerzon 1975) are deliberately
74 // omitted: the STONE tetrahedral loudspeaker amplifier handles
75 // frequency-domain correction downstream.
76 //
77 // References:
78 // - Gerzon, "Ambisonics. Part two: Studio techniques" (1975)
79 // - Malham, "Space in Music -- Music in Space" (1998)
80 //
81 bamodulex(a0,a1,a2,a3) = lfu, rfd, rbu, lbd
82 with {
83     lfu = (a0 + a1 + a2 + a3) / 2;
84     rfd = (a0 - a1 - a2 + a3) / 2;
85     rbu = (a0 - a1 + a2 - a3) / 2;
86     lbd = (a0 + a1 - a2 - a3) / 2;
87 };
88 //
89 //           First-order AmbiX rotation
90 //
91 // Applies 3D rotation to a first-order AmbiX signal.
92 // Rotation is applied in intrinsic order: Roll -> Pitch -> Yaw.
93 // The resulting matrix is R = Rz(yaw) * Ry(pitch) * Rx(roll).
94 //
95 // Roll (X-axis) -- lateral tilt (left-right vertical plane)
96 // Pitch (Y-axis) -- forward-backward tilt (vertical plane)
97 // Yaw (Z-axis) -- horizontal rotation (azimuth)
98 //
99 // Channel W (omnidirectional) is unaffected by rotation.
100 //
101 // Arguments:
102 // yaw, pitch, roll -- rotation angles in radians
103 // a0, a1, a2, a3 -- input channels in ACN order (W, Y, Z, X)
104 //

```

```

105 rotateYPR(yaw, pitch, roll, a0, a1, a2, a3) =
106     a0, a1_ypr, a2_ypr, a3_ypr
107 with {
108     // Roll (X-axis): rotates Y and Z
109     a1_r = a1*cos(roll) - a2*sin(roll);
110     a2_r = a1*sin(roll) + a2*cos(roll);
111     a3_r = a3;
112     // Pitch (Y-axis): rotates Z and X
113     a1_rp = a1_r;
114     a2_rp = a2_r*cos(pitch) + a3_r*sin(pitch);
115     a3_rp = -a2_r*sin(pitch) + a3_r*cos(pitch);
116     // Yaw (Z-axis): rotates Y and X
117     a1_ypr = a1_rp*cos(yaw) - a3_rp*sin(yaw);
118     a2_ypr = a2_rp;
119     a3_ypr = a1_rp*sin(yaw) + a3_rp*cos(yaw);
120 };
121 //
122 //----- UHJ C-FORMAT ----
123 // AmbiX (ACN/SN3D) -> UHJ C-format L,R,T,Q.
124 // Gerzon, "Ambisonics in Multichannel Broadcasting and Video", JAES 1983.
125 // Quadrature all-pass pair re-derived analytically (SEAM-LTM x2uhj, 2026);
126 // max phase error 1.36 deg over 20 Hz-20 kHz. Coefficients are the single
127 // source of truth in seam-ltm plugins/x2uhj/source/x2uhj_coeffs.h.
128 // NB canonical structure: every component passes the real network hR; only the
129 // j-terms use the imaginary network hI. The 90 deg is the DIFFERENCE between
130 // the two matched networks. Earlier drafts leaving the real path unfiltered
131 // are superseded.
132 //
133 // 2nd-order RBJ all-pass parameterised by (frequency, Q):
134 apuhj(f,Q) = fi.tf2(a2, a1, 1.0, a1, a2)
135 with {
136     w = 2.0*ma.PI*f/ma.SR;
137     alpha = sin(w)/(2.0*Q);
138     n = 1.0 + alpha;
139     a1 = -2.0*cos(w)/n;
140     a2 = (1.0 - alpha)/n;
141 };
142 // Matched quadrature pair (3 sections each):
143 hRuhj = apuhj(141.887012,0.201947) : apuhj(671.745773,0.212154) : apuhj
144     (18654.238494,0.303097);
145 hIuhj = apuhj(24.003923,0.309038) : apuhj(2991.937233,0.384766) : apuhj
146     (3219.986192,0.096342);
147 // AmbiX ACN inputs in order (W,Y,Z,X) -> UHJ (L,R,T,Q):
148 x2uhj(aW,aY,aZ,aX) = L,R,T,Q
149 with {
150     W = aW/sqrt(2.0); X = aX; Y = aY; Z = aZ; // ACN/SN3D -> FuMa
151     Wr = hRuhj(W); Xr = hRuhj(X); Yr = hRuhj(Y); Zr = hRuhj(Z);
152     Wi = hIuhj(W); Xi = hIuhj(X);
153     S = 0.9396926*Wr + 0.1855740*Xr;
154     D = (-0.3420201*Wi + 0.5098604*Xi) + 0.6554516*Yr;
155     L = 0.5*(S + D);
156     R = 0.5*(S - D);
157     T = (-0.1432*Wi + 0.6512*Xi) - 0.7071*Yr;
158     Q = 0.9772*Zr;
159 };
160 //process = x2uhj;
161 //
162 //----- MONO -> AMBIX (HAAR) ----
163 // Mono to first-order AmbiX via the Haar wavelet decomposition
164 // (sdw.haarmn from seam.dwt.lib). Channel 0 (W) is the passthrough; the
165 // QMF sub-bands form the directional components. Rotation, if any, is
166 // applied separately with rotateYPR. SEAM-LTM M2XHGR plugin.
167 m2xhgr = sdw.haarmn(1);
168 //process = no.noise : m2xhgr;
169 //
170 //----- STEREO -> AMBIX (HAAR) ----
171 // Stereo (L,R) to first-order AmbiX via Haar, with a divergence control.
172 // Each channel is Haar-encoded (m2xhgr), rotated by +/-divergence, summed,

```

```

171 // then globally rotated by yaw. divergence/yaw/pitch/roll in radians.
172 // SEAM-LTM LR2XHGR plugin.
173 lr2xhgr(divergence, yaw, pitch, roll) =
174   par(i, 2, m2xhgr) :
175     rotateYPR(divergence, pitch, roll),
176     rotateYPR(0 - divergence, pitch, roll)
177   :> si.bus(4) :
178     rotateYPR(yaw, 0, 0);
179 //process = (no.noise, no.noise) : lr2xhgr(ma.PI/2, 0, 0, 0);
180 //
181 //----- A-FORMAT -> AMBIX (ACN) ---
182 // Tetrahedral microphone A-format (LFU,RFD,RBU,LBD) -> First-order AmbiX
183 // (ACN: a0=W, a1=Y, a2=Z, a3=X). Inverse of bamodulex; the Hadamard/2
184 // matrix is involutory ( $M^2 = I$ ), so the coefficients are identical.
185 // SEAM-LTM abmodulex plugin (TETRAREC front-end).
186 abmodulex(lfu,rfd,rbu,lbd) = a0, a1, a2, a3
187   with {
188     a0 = (lfu + rfd + rbu + lbd) / 2;
189     a1 = (lfu - rfd - rbu + lbd) / 2;
190     a2 = (lfu - rfd + rbu - lbd) / 2;
191     a3 = (lfu + rfd - rbu - lbd) / 2;
192   };
193 //process = sno.multipink(4,0.5) : abmodulex;

```